$$Ax = b \to x = Gx + Qb$$

Section 1 Classical Iterative Methods

$$\begin{aligned} f(x) = 0 \to x &= x - f(x) \\ (1) \to qx &= qx - f(x) \end{aligned}$$

\begin{itemize} \item[(2)] $\to qx + g(x)$

$$= qx - (f(x) - g(x))$$

\item[(3)] $\to qx + kg(x)$

$$= qx - k(f(x) - g(x))$$

\end{itemize}

$$x^{k+1} = Gx^k + d$$

$$\begin{aligned} Qx &= Qx - (Ax - b) \\ &= (Q - A)x + b \end{aligned}$$

$$Qx_{k+1} = (Q - A)x_k + b$$

$$\begin{aligned} Qx + Bx &= Qx - ((A - B)x - b) \\ &= (Q - A + B)x + b \end{aligned}$$

$$(Q + B)x_{k+1} = (Q - A + B)x_k + b$$

$$(Q + kB)x = Qx - k((A - B)x - b)$$

\section*{Section 1 Classical Iterative Methods}

$$x^{k+1} = Gx^k + d$$

\begin{itemize} \item[] −Jacobi iteration \item[] −Gauss–Seidel iteration \item[] −SOR iteration \end{itemize}

The necessary and sufficient condition for convergence is $\rho(\mathrm{G}) < 1$

If $\|\mathrm{G}\| < 1$, the method converges If A is strictly diagonally dominant, the method converges ($0 < \omega <= 1$)

$$\begin{cases} x_1^{(k+1)} = -\frac{a_{12}}{a_{11}}x_2^{(k)} - \frac{a_{13}}{a_{11}}x_3^{(k)} - \cdots - \frac{a_{1n}}{a_{11}}x_n^{(k)} + \frac{b_1}{a_{11}} \\ x_2^{(k+1)} = -\frac{a_{21}}{a_{22}}x_1^{(k)} - \frac{a_{23}}{a_{22}}x_3^{(k)} - \cdots - \frac{a_{2n}}{a_{22}}x_n^{(k)} + \frac{b_2}{a_{2n}} \\ \cdots\cdots\cdots\

recurrences, requires only 1 matvec per iteration, and uses $O(n)$ storage.
    - **GMRES (Generalized Minimal Residual method, Saad & Schultz, 1986)**: applicable to general nonsymmetric nonsingular matrices; builds an orthonormal basis via the Arnoldi process and minimizes the 2-norm of the residual, with storage growing with the iteration count.

- **Key insight**: CG exploits symmetry to achieve short recurrences, whereas GMRES makes no symmetry assumption on A and therefore requires long recurrences and more storage. **This is the fundamental trade-off between "generality" and "efficiency".**

2.2 The Arnoldi Process

Teaching Content:

(I) Goal of the algorithm: given $A \in \mathbb{R}^{n\times n}$ and an initial vector v_1 (usually taken as $v_1 = r_0/\|r_0\|_2$), construct an orthonormal basis $\{v_1, v_2, \ldots, v_m\}$ spanning $\mathcal{K}_m(A, v_1)$.

(II) Pseudocode of the Arnoldi algorithm

```
Algorithm Arnoldi(A, v_1, m)
    for j = 1, 2, ..., m do
        w = A v_j                      // matrix-vector product
        for i = 1, 2, ..., j do
            h_{i,j} = w^T v_i          // project onto existing basis vectors
            w = w - h_{i,j} v_i        // orthogonalization (modified Gram-Schmidt)
        end for
        h_{j+1,j} = ||w||_2            // length of the new direction
        if h_{j+1,j} == 0 then         // converged (happy breakdown)
            m = j; break
        end if
        v_{j+1} = w / h_{j+1,j}        // normalize to obtain the new basis vector
    end for
    return V_m = [v_1, ..., v_m], H_{m+1,m} = [h_{i,j}]
```

(III) Core properties of the Arnoldi process

After m Arnoldi steps, one obtains the factorization relation

$$AV_m = V_{m+1}\tilde{H}_m,$$

where $V_m \in \mathbb{R}^{n\times m}$ has orthonormal columns ($V_m^T V_m = I_m$), and $\tilde{H}_m \in \mathbb{R}^{(m+1)\times m}$ is an upper Hessenberg matrix (only $h_{i,j}$ with $i \le j+1$ are nonzero).

This factorization compresses the action of the high-dimensional matrix A into the low-dimensional upper Hessenberg matrix $\tilde{H}_m$, and is the key to deriving the GMRES algorithm.

2.3 The GMRES Algorithm (15 minutes)

Teaching Content:

(I) Reformulation of the problem

Let $x_m = x_0 + V_m y$ ($y \in \mathbb{R}^m$); then the residual is

$$r_m = b - Ax_m = r_0 - AV_m y = \beta v_1 - V_{m+1}\tilde{H}_m y,$$

where $\beta = \|r_0\|_2$ and $v_1 = r_0/\beta$.

Using the orthonormal columns of V_{m+1}, the residual norm simplifies to

$$\|r_m\|_2 = \|\beta e_1 - \tilde{H}_m y\|_2,$$

where $e_1 = (1, 0, \ldots, 0)^T \in \mathbb{R}^{m+1}$.

Core conclusion: the optimal solution of GMRES at step m is equivalent to solving the $(m+1) \times m$ least-squares problem

$$y_m = \arg\min_{y \in \mathbb{R}^m} \|\beta e_1 - \tilde{H}_m y\|_2.$$

(II) Solving the upper Hessenberg least-squares problem

The key point is that the upper Hessenberg structure of $\tilde{H}_m$ allows the problem to be solved recursively through a sequence of **Givens rotations**, avoiding recomputing the QR factorization from scratch at every step:

1. After the j-th Arnoldi step produces the new column of $\tilde{H}_j$, apply the already computed Givens rotations to the new column;
2. Generate a new Givens rotation G_j to eliminate the subdiagonal entry $h_{j+1,j}$ of $\tilde{H}_j$;
3. Accumulate the rotations into the right-hand side vector βe_1;
4. Check $|(\beta e_1)_{j+1}|$ (the last component after rotation) — this is the estimate of the residual norm at the current step.

(III) The complete GMRES algorithm

```
Algorithm GMRES(A, b, x_0, m_max, tol)
    r_0 = b - A x_0
    beta = ||r_0||_2; v_1 = r_0 / beta
    for j = 1, 2, ..., m_max do
        // Arnoldi step j
        w = A v_j
        for i = 1 to j do
            h_{i,j} = w^T v_i
            w = w - h_{i,j} v_i
        end for
        h_{j+1,j} = ||w||_2; v_{j+1} = w / h_{j+1,j}

        // apply Givens rotations to the new column
        for i = 1 to j-1 do
            rotate(h_{i,j}, h_{i+1,j}, c_i, s_i)
        end for
        (c_j, s_j) = givens(h_{j,j}, h_{j+1,j})
        h_{j,j} = c_j * h_{j,j} + s_j * h_{j+1,j}
        h_{j+1,j} = 0
        g_{j+1} = -s_j * g_j
        g_j = c_j * g_j

        // check convergence
        res_norm = |g_{j+1}|
        if res_norm < tol * ||b|| then
            // back substitution to solve for y
            solve H_j(1:j,1:j) * y = g(1:j)   (upper triangular)
            x = x_0 + V_j y
            return x
        end if
    end for
    // not yet converged at the maximum number of steps; a restart is needed
    x = x_0 + V_{m_max} y; x_0 = x; goto start
```

(IV) Restarting strategy (Restarted GMRES)

- **Problem**: GMRES stores all m Arnoldi vectors ($O(mn)$); when m is too large, the memory requirement becomes unaffordable.
- **Solution**: use the restarted version GMRES(m) — restart every m steps, restarting the Arnoldi process with the current approximate solution as the new initial guess.
- **Cost**: restarting discards superlinear convergence information and may cause stagnation for certain problems (e.g., matrices with clusters of small eigenvalues).
- **Practical recommendation**: m is usually taken as 20–50, and the larger the better within the available memory.

(V) Convergence properties of GMRES

- **In exact arithmetic**: GMRES converges to the exact solution in at most n steps (when $m = n$, the last row of $H_{n+1,n}$ is entirely zero, i.e., a "happy breakdown").
- **Convergence rate**: determined by the spectral distribution of A (the "clustering" of the eigenvalue set). Convergence is fast when the eigenvalues are tightly clustered, and slow when they are scattered and close to the origin.
- **Key insight**: the condition number $\kappa(A)$ is not a reliable indicator of the GMRES convergence rate — a matrix with a large condition number but clustered eigenvalues may converge faster than one with a small condition number but a scattered spectrum.
- **Importance of preconditioning**: the convergence of GMRES strongly depends on preconditioning — this is the core content of Session 3.

Theorem: given $\boldsymbol{A}$ is symmetrical and positive definite, and its eigenvalues $\boldsymbol{\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n > 0}$ The sequence gives by the steepest gradient method is convergent

$$\lim_{k\to\infty} x^{(k)} = x^* = A^{-1}b$$

and holds

$$\left\|x^{(k)} - x^*\right\|_A \leq \left(\frac{\lambda_1 - \lambda_n}{\lambda_1 + \lambda_n}\right)^k \left\|x^{(0)} - x^*\right\|_A$$

$$\|x\|_A \triangleq \sqrt{x^T A x}$$

- If $\lambda_1 \gg \lambda_n$, the convergence is very slow

3.1 Basic Concepts of Preconditioning (10 minutes)

Teaching Content:

(I) Why preconditioning is needed

The convergence of GMRES depends on the spectral properties of the coefficient matrix A. For many practical engineering problems, the condition number of A is very large or its eigenvalues are widely scattered, causing GMRES to converge extremely slowly. Preconditioning aims to construct a matrix M (the preconditioner) such that $M^{-1}A$ (left preconditioning) or AM^{-1} (right preconditioning) has better spectral properties.

(II) Preconditioning formulations

- **Left preconditioning**: solve $M^{-1}Ax = M^{-1}b$. The residual norm corresponds to the preconditioned system and may deviate from the original residual.
- **Right preconditioning**: solve $AM^{-1}y = b$, with $x = M^{-1}y$. GMRES uses the original residual norm as the stopping criterion, which is safer. GMRES usually adopts right preconditioning.

After preconditioning, the Arnoldi relation becomes

$$(AM^{-1})V_m = V_{m+1}\tilde{H}_m.$$

(III) The ideal preconditioner

Obviously $M = A$ is optimal (in this case the system is solved exactly), but constructing A^{-1} costs as much as the original problem. A practical preconditioner must strike a balance between **construction cost** and **preconditioning effectiveness**. The incomplete LU factorization is precisely the classical scheme that achieves this balance.

3.2 Incomplete LU Factorization (20 minutes)

Teaching Content:

(I) From complete LU to incomplete LU

A complete LU factorization decomposes A as $A = LU$, where L is unit lower triangular and U is upper triangular. For sparse matrices, a large number of **fill-ins** appear in L and U — that is, entries that are zero in the original matrix A but become nonzero after the factorization. Fill-in causes an explosion in storage and computation.

The core idea of the **incomplete LU factorization** (ILU) is: **selectively discard fill-ins during the factorization so as to preserve the sparsity of L and U**, such that $M = LU \approx A$ while the construction cost remains manageable.

(II) ILU(0): the zero-fill-level ILU

ILU(0) is the simplest version: **nonzero entries of L and U are allowed only within the nonzero pattern of A**, and all fill-ins are discarded.

3.3 The Complete ILU-Preconditioned GMRES Algorithm (15 minutes)

Teaching Content:

(I) Right-preconditioned GMRES algorithm

```
Algorithm PGMRES(A, b, M, x_0, m_max, tol)
    // M = L*U is the ILU preconditioner
    r_0 = b - A x_0
    beta = ||r_0||_2; v_1 = r_0 / beta
    for j = 1, 2, ..., m_max do
        // preconditioning operation z = M^{-1} v_j
        Solve L z_tmp = v_j    // forward substitution
        Solve U z = z_tmp      // back substitution

        // perform Arnoldi on the preconditioned vector
        w = A z
        for i = 1 to j do
            h_{i,j} = w^T v_i
            w = w - h_{i,j} v_i
        end for
        h_{j+1,j} = ||w||_2; v_{j+1} = w / h_{j+1,j}

        // Givens rotations and convergence check (as in the previous section)
        ...
    end for
    x = x_0 + Z_m y          // Z_m = [z_1, ..., z_m]
    return x
```

Key differences from standard GMRES:

1. Before each Arnoldi iteration, one **forward substitution + back substitution** is required ($O(\mathrm{nnz}(L+U))$) — this is the main cost of preconditioning;
2. The number of iterations required for convergence is greatly reduced (typically by 1–2 orders of magnitude);
3. The approximate solution is built from a linear combination of the z vectors rather than the v vectors.

(II) Computational cost analysis

Operation	Complexity per iteration
SpMV (Az)	$O(\mathrm{nnz}(A))$
ILU forward + back substitution (L, U)	$O(\mathrm{nnz}(L+U))$
Arnoldi orthogonalization (MGS)	$O(jn)$

The trade-off of preconditioning overhead: although each Arnoldi iteration adds the cost of a forward/back substitution, the iteration count drops from hundreds to fewer than ten, so the **total computation time is usually greatly reduced**.

(III) An intuitive interpretation

ILU preconditioning is equivalent to performing a "coarse-grained approximate inversion" of A before applying GMRES. This approximate inverse $M^{-1} \approx A^{-1}$ maps the eigenvalues of A to the neighborhood of 1, so that GMRES needs only a small correction on a system that is "almost already solved".

Numerical visualization: in the examples, plot a comparison of the eigenvalue distributions before and after ILU preconditioning — the preconditioned eigenvalues should be clearly clustered around 1.